

Mini Secure Fintech Wallet

Security and System Design Report

Course: Secure System Design (MS-CY) - Assignment 1

Prepared by:

Syed Mohid Ul Hassan - 26I-7709

Muhammad Akash Afzal - 26I-7807

Technology Stack: React, TypeScript, NestJS, Prisma ORM, SQLite

**National University of Computer & Emerging Sciences (FAST - NUCES)
Islamabad**

1. System Overview

The Mini Secure Fintech Wallet is a small, working digital wallet application. This app lets multiple users registered create an account, log in, check their balance, and send money to each other, and also view their past transaction history. We built it as a desktop web application, using React with TypeScript on the frontend and NestJS on the backend, with Prisma and SQLite handling the database.

The interface was designed very minimalistic and cleanly, as the main goal of the assignment is to learn what can really go wrong in such a system and develop controls that actually solve the problems. All the controls in this report are in place because they were identified as a weakness first. There was no list of security technologies and then we attempted to squeeze them into this project; each control is related to something we discovered by examining our own application honestly.

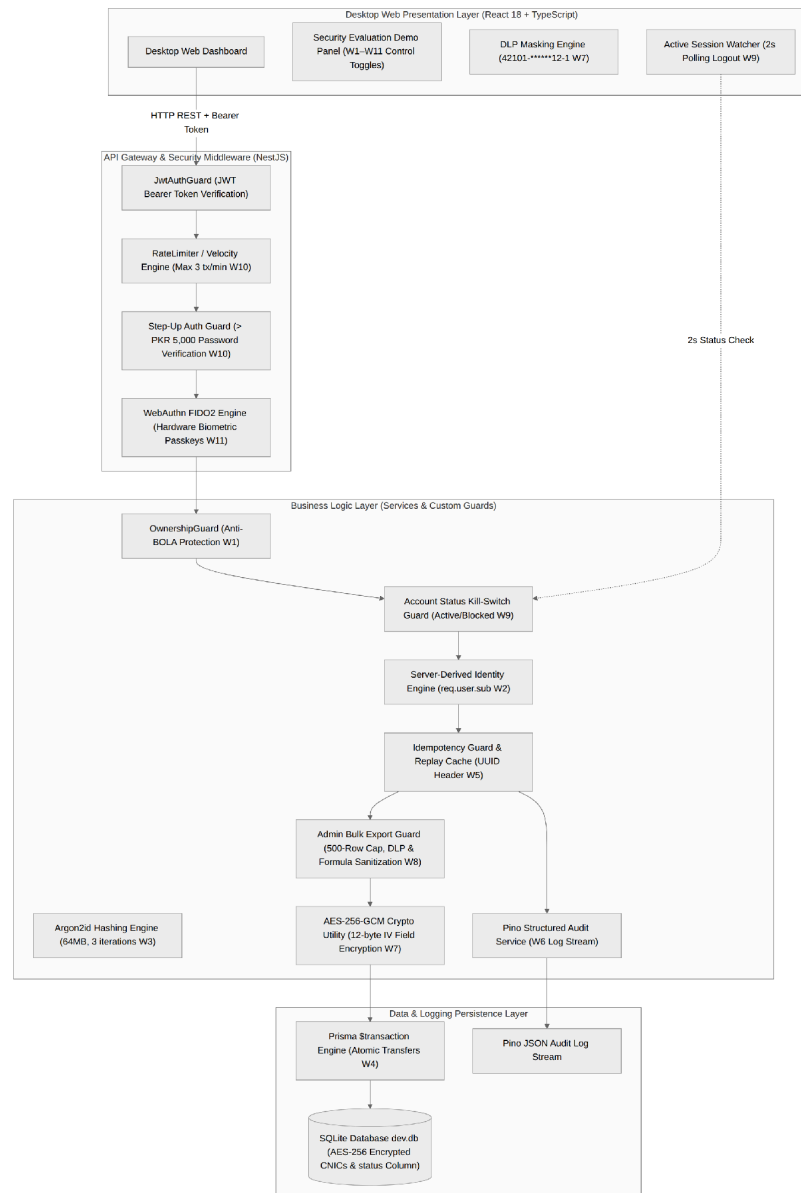
By mapping with the concepts discussed in the class, we identified eleven weaknesses that we think are relevant to our own implementation, labelled W1 through W11. The eleventh, W11, is relying on a password alone to prove identity, which we addressed by adding passwordless login using WebAuthn / FIDO2 passkeys as an optional alternative. We included this since it immediately builds on phishing-resistant authentication, a topic we learned in class, and it was something we could easily implement into the project deadline.

Core Functional Scope

- Customer registration, including basic identity information (name, email, CNIC).
- Login and logout, using password-based authentication with an optional passkey login.
- Viewing the current wallet balance.
- Transferring funds between two registered users, with the sender's balance going down and the receiver's balance going up.
- Viewing transaction history, including sender, receiver, amount, date/time, and status.
- A small admin view, used to demonstrate account-blocking and bulk-export controls and view other statistics related to the app.

2. Architecture Diagram

The diagram below shows how a request moves through the system, layer by layer, and where each security control sits. We grouped the system into four layers: the presentation layer (what the user sees and interacts with in the browser), the API gateway layer (the first checks every request has to pass), the business logic layer (where the actual rules of the wallet live, including most of our security guards), and the persistence layer (the database and the audit log).



Mini Secure Fintech Wallet

System & Security Architecture: layered request flow and control placement (W1 to W11)

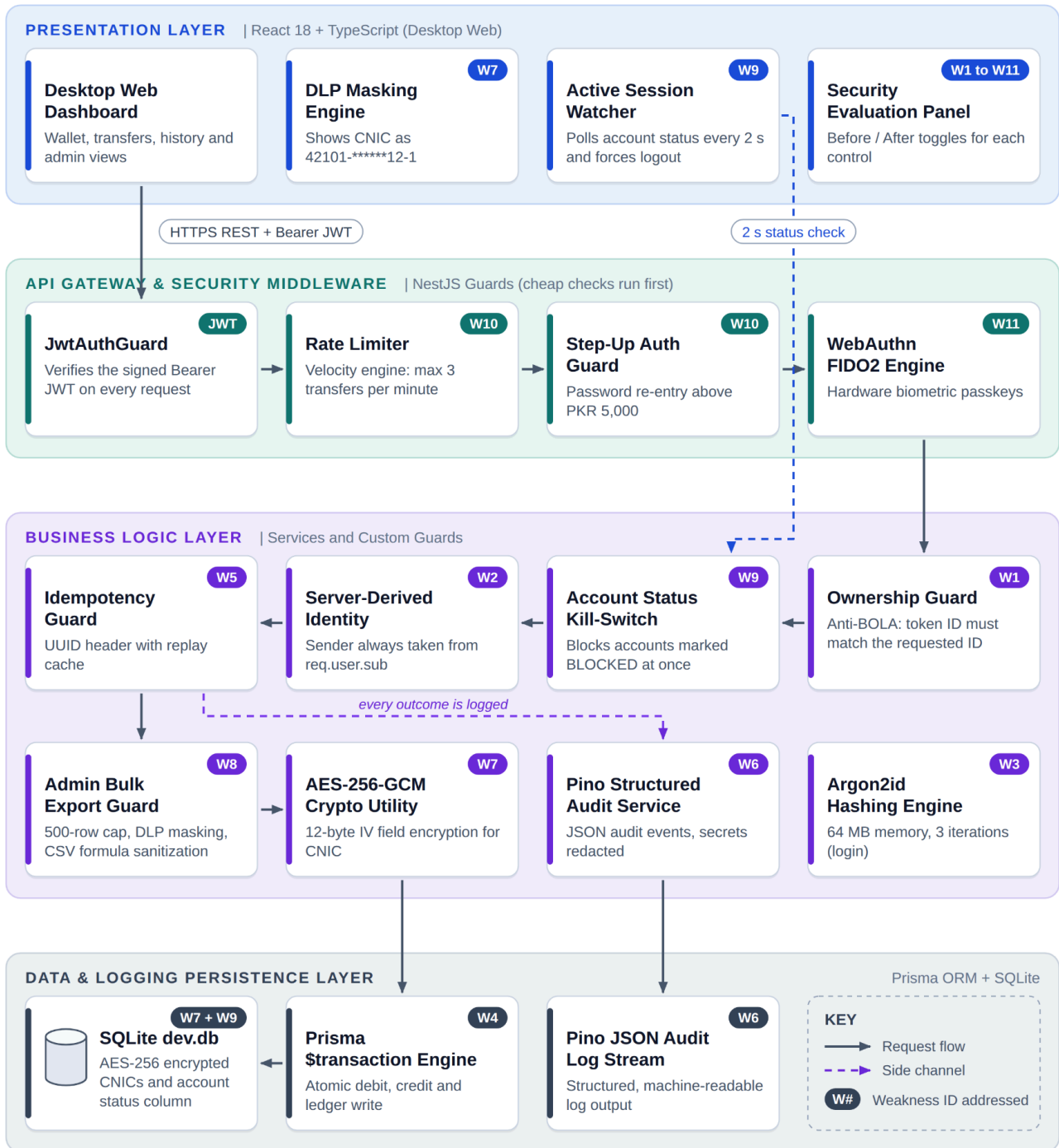


Figure 1 — Request flow and security control placement across the four layers of the system.

All requests sent from the browser include a JWT bearer token. The order in which the backend checks things: first: is the token itself valid? (JwtAuthGuard); second: are requests coming in too fast? (the rate limiter); third: (only for larger transfers): has the user re-entered his/her password? (the step up guard). This is why we ordered the checks like that - if the server receives a lot of junk requests, they will be rejected early, since the cheap checks are done first and the expensive ones, such as a database lookup, are not.

Upon reaching the business logic layer, the request goes through a number of built-in guards: one to ensure that users can only view an account's wallet if they own it; one to ensure that an account is never allowed to view a token if it has been blocked; and finally, one to ensure that a user never gets the money from an account's wallet that has been sent by the client, but rather only from the verified wallet. From there, the real database manipulation takes place within one atomic transaction and all results (success or failure) are recorded in a structured audit log.

3. Major Assets

Before deciding on any controls to choose, we first listed what in our system is actually we should really protect, and why. This list guided every weakness we looked for afterwards.

Asset	What It Is	Why It Matters
User credentials	Customer passwords and their Argon2id hashes.	If these leak or are guessable, an attacker can log in as the customer and that could be dangerous..
Customer identity information	Full name, email address, and CNIC number.	This is personal data. If it leaks, customers may face real-world identity theft risk, not just an inconvenience, so we have to protect that.
Wallet balance	The amount of money each customer holds.	This is the core financial value in the system - the whole point of a wallet is that this number should be working accurately and stays correct.
Transaction records	Who sent what, to whom, and when.	These records are basically the evidence of what happened in the system. If they can be altered or faked, nothing in the system can be trusted and reputation will be lost..
Account status	Whether an account is active or blocked.	If any blocked account can still move money, blocking is useless then..
Audit logs	A record of logins, transfers, and security-relevant events.	Without logs, there is no way to investigate what happened after something goes wrong so that's why it is important..
Application availability	The wallet's ability to respond to legitimate requests.	If the service can be flooded or exhausted, real customers cannot use their own money so we have to keep our app secure from these attacks.
Account identity (IBAN)	The unique wallet account number (PK99SWAL...) assigned to each customer.	If account numbers could be manipulated or spoofed, money could be transferred to the wrong receiver.

4. Identified Security Weaknesses and Their Possible Impact

We have walked through the application, function by function and asked ourselves the kinds of questions the assignment demands: Can a user view another user's data? Is it possible to spoof who a transfer is coming from? What if a request is sent twice? From that process we came up with 11 weaknesses that we actually had (or would have had, if we hadn't addressed them) in our own implementation.

ID	Weakness	Possible Impact If Exploited
W1	The balance and history endpoints trusted a user ID sent in the request URI and did not verify that the user ID was actually the user's who made the request.	You can change an ID in the request to see the customer's money balance or a list of his purchases without having to log on as the owner.
W2	Instead of determining the sender, the transfer endpoint received a "sender" field from the client.	Attackers may be able to request a bank transfer to himself, on behalf of the other person, and take that person's money without their knowledge.
W3	The passwords might have been stored in an insecure manner, and error messages when logging in could have indicated if an e-mail was registered.	If a database leak occurs, it would reveal easily guessable passwords, and the login form could be used to determine which emails are actually customer emails.
W4	The transfers were initially not carried out in a single transaction but as two transactions (debit, then credit).	If the server crashes in between the two writes, money might not be received by the receiver and it might be lost from the sender's wallet.
W5	The transfer request could never be processed twice, in case a retried network request occurred.	A user may end up being charged twice for the same transfer due to a network hiccup or a double click.
W6	No security-related events (logins, failed logins, transfers, and blocked attempts) were being captured anywhere.	If something did occur, it would be impossible to determine what really happened after the event.
W7	The CNIC numbers were being stored as plain text and any user's own statement could be exported without the need for additional checks.	Anyone who could obtain the database file, or the exported file would have raw national identity numbers.
W8	There was no restriction or volume limit on a bulk export (used by an admin to pull many customers' records at once); there was no CSV formula injection or masking protection.	This is the exact same principle that is used for real-world breaches such as the Santander incident in 2024, which allowed for the withdrawal of all the customer information from just one account. An unsanitized export might also allow a malicious value within a cell to be evaluated as a formula when it was opened in Excel.
W9	If a customer was logged in, they could not be immediately blocked in the database.	If the account was compromised or fraudulent, it might continue to transfer funds for as long as 15 minutes after blocking, but until the token expires.
W10	Transfers were unlimited and large transfers required only standard login.	A script might trigger transfers one after another or a single compromised session may be a one-shot to transfer a significant amount of funds.
W11	Login was dependent on a password only, and so a stolen, reused, or phished password was sufficient to compromise a login.	If an attacker had deceived a customer into entering their password on a fake logon screen, or if an attacker used a password that was leaked before, then they would be able to log in as that customer.

6. Proposed Security Controls

For every weakness above, we picked one control that addresses it directly, rather than reaching for the most impressive-sounding technology available. The table below gives the short version; Section 7 explains the reasoning behind each choice, including what we considered and decided not to use.

ID	Proposed Control
W1	An ownership guard to verify that the logged in user has an ID that matches that of the request before returning any data.
W2	Getting the sender of a transfer only from the verified login token and never from anything sent by the client.
W3	A single generic error message for any failed login, along with argon2id password hashing. Making debit and credit of a transfer in one atomic database transaction.
W4	Supporting both the debit and credit of a transfer in a single atomic database transaction.
W5	An idempotency key is included in each transfer request, so each time the same request is made, the same result is returned and not executed again.
W6	A detailed audit log that logs all login attempts and all transfer attempts, but not secrets.
W7	Masking CNIC field on screen and masking on exports, encrypting the CNIC field at rest with AES-256-GCM.
W8	Only giving the admin user access to bulk export, limiting the number of records in an export, masking personal information from the output, and sanitizing cells to prevent CSV formula injection.
W9	Saving the status of the account straight into the database, and having the front-end check this every two seconds and log the user out right away if the status changes to blocked.
W10	A rate limit on the transfer endpoint and a step up check (re-enter your password) for transfers that exceed a certain limit.
W11	In addition to the password, WebAuthn / FIDO2 passkey login (fingerprint, face unlock or a hardware security key) was provided to customers as an additional method to prove their identity that is phishing resistant.

7. Explanation of Implemented Controls

This is where we describe what we actually did think and did - not what we built, but how we built, and why we built it that way, and in some cases, why we didn't build it some other way. A control can't be of any value unless we can give a reason for it being suitable for the weakness it is intended to address.

W1: Ownership Guard (Object-Level Authorization)

User ID is compared against the ID in the verified JWT, if they do not match, a 403 response is passed. This is because "logged in" is not sufficient to be treated as proof of permission to access all data, and that's exactly how this kind of weakness comes about in the first place.

We did not create a more complex permissions structure such as full role hierarchies or attribute based access control as our application has only two types of users - a customer and an administrator and a simple check of ownership is sufficient for all the cases that we actually have. A more complex system would not have provided any greater protection for this project.

W2: Server-Derived Sender Identity

We use the verified token's subject field as the only truth in the world of sending money, and we do not even look at what the client sends in the request body in terms of sender information. We use this because a client can postpone its input to the server before it reaches the server, and if we rely on a field such as "senderId" in the request, the attacker can set it to be whatever they please - and thus decide which money moves.

We didn't include a separate confirmation step in this particular weakness (such as re-typing the sender's account), since the fix cannot have such a step - once the sender identity can only be obtained from the token, there simply isn't a field left for an attacker to mess with.

W3: Argon2id Password Hashing and Anti-Enumeration

We have stored the password in hash format using Argon2id and return the same error message ("invalid email or password") for both missing emails and incorrect passwords. The argument in favour of using Argon2id instead of an older algorithm such as plain SHA-256, or even bcrypt is that it is memory hard, so that it is deliberately difficult to break using a gpu, hence it is the algorithm that is now recommended for new projects by OWASP.

We also looked at bcrypt, which is still regarded as acceptable, but for very little additional implementation effort we chose Argon2id, which provided better protection. Then we didn't have full multi factor authentication (e.g. SMS one-time codes) as the default way to log in, as that would have required a third party SMS service, and that's beyond what we could reasonably set up and demonstrate for this assignment. We opted for a stronger alternative to a password instead, with WebAuthn passkeys (explained separately below), which we made an optional feature.

W4: Atomic Database Transactions

We took advantage of Prisma's transaction functionality to encapsulate the debit and the credit of the transfer in a single database operation, such that both the debit and the credit are either performed or not performed. We had to do this since there was a gap between the two separate states for which a crash or a disconnect could cause us to lose money - to one account without transferring it to the other.

We didn't use a distributed-transaction system (the kind that a large multi-database bank may use), because all of our applications are run against a single database. Here, there is a need for a single atomic transaction, not a distributed one, because we don't have this problem.

W5: Idempotency Keys

We applied a custom unique key per transfer request generated with a client. The server keeps track of those keys that it already has seen and when the same key comes again, it returns the same value that it had previously returned without transferring money again. This was done because retries are a normal occurrence in networks, whether it's a slow connection, a double click or an automatic retry, the same request could arrive twice and each occurrence would remove money.

We have not used a time based deduplication approach (such as "block any identical transfer in 5 seconds"): such a rule can prevent two "true" distinct transfers from passing through, if they appear at the same time. An idempotency key that is tied to a specific request is more specific since it will only catch true duplicates of the same request, not unrelated requests that just happen to be the same.

W6: Structured Audit Logging

We log all login attempts, all transfer attempts, all attempts that were rejected, and all admin actions (but we don't log secrets such as passwords or full tokens, and this is not a bug for us). This is because if there is no record of the incident, there is no way to answer simple questions afterward such as: "Who logged in, when, and what did they attempt to do?"

We didn't build a full external log monitoring system (the sort of SIEM system that we've talked about in class, such as Splunk or Sentinel) because that sort of infrastructure isn't really useful for a single-developer demo project. What we ended up with is the building blocks of such a system providing structured logs which could be later ingested into a real monitoring system: clean, structured logs.

W7: Field-Level Encryption and Masking for CNIC

We've encrypted the CNIC field before inserting it into the database, using AES-256-GCM with a randomly generated IV for each individual record (otherwise we'd be trivially breaking its security guarantees, so we took care of that). If the value has to be displayed to a user it is decrypted before it is displayed and then masked prior to it being exported to the screen, so that the display of a value is more or less a partially hidden number than the actual one.

We thought about storing the value as a "42101-*****12-1" directly in the database, but we didn't do that, as this would strip the bank of the ability to carry out identity verification or to respond to a subsequent request for that verification. Encryption of the value ensures that it can be recovered for valid uses yet is unreadable to those who have just the raw database file.

We didn't store the encryption key in a managed cloud key service, such as AWS KMS or a hardware security module, because we don't have such infrastructure for this assignment. Rather, the key is taken from an environment variable, as we did mention it as a limitation in Section 11.

W8: Admin Bulk Export Controls

For the bulk export feature, we used the combination of the following features: Bulk export can only be triggered from an admin-role account, The maximum limit for each bulk export is 500 records, Personal fields such as CNIC and email are masked in the output, Every cell that is exported is sanitized from CSV formula injection before the file is written. We have used this combination because this is the mode of attack that occurred in the real breaches that we have studied in class, such as the case of Santander which in 2024 was able to get a very high number of customer records in a single request from a compromised account with no role check, no limit and no masking.

The formula-injection sanitisation requires some explanation of its own, as it's different from the others. If a value in our database started with a character such as =, +, -, or @, and the value was entered directly into a CSV file, a

program such as Excel or Google Sheets would attempt to execute it as a formula when the file is opened, rather than just interpreting it as text. We did not want to take that for granted as part of the name or note/references of transactions are supplied by the user. Our export function will test first all cells before dumps are written, and if any of those cell values begins with one of the above characters, it will add a single quote to the front of the value so that spreadsheet software treats it as a plain value rather than a formula.

We did not implement an entire approval process, where a second person needs to approve each export before it is run, as this is something a larger organisation would require, and not a two-person student project. The restrictions on roles, the hard caps on volume and the masking and sanitization of formula only directly seal the distinct holes we were able to see, and without implementing a process that we wouldn't be able to meaningfully demonstrate.

W9: Account Status Kill-Switch

We had a status column in the account (active or blocked), and we had a front end check that would check the status every two seconds while in login. When logging in to a sitelock that has been blocked by an admin, the next poll sees the change and will force a logout, even if the user's logging in token has several minutes remaining before it expires automatically.

We didn't reduce its natural lifetime to one minute or something short, since if this is supposed to be something convenient, then forcing the user to login otherwise continuously is not a good idea. We polled the account status individually, thus getting fast and reliable blocking without any hassle for normal and honest use of the token.

W10: Rate Limiting and Step-Up Authentication

Here we had two controls: a rate limit to ensure that there are 3 transfers per minute per user and a step-up check to prompt for the account password again before a third transfer occurred once the limit is reached. We implemented the rate limit because there should be no single scripting that can get a ton of cash, and we added the step-up check because if a session was stolen or left unattended, it should not be enough to itself to move a large amount of cash.

We didn't require a second authentication factor (such as an SMS code) for all transactions, irrespective of how small, as this would have slowed down, and made even normal transferring very tedious for little reward. The same principle of risk-based, adaptive authentication that was discussed in class - with the additional twist that the extra check can only be used if the risk is higher, if the transaction is larger.

W11: WebAuthn / FIDO2 Passkey Login

We applied the WebAuthn standard to enable a user to register a passkey by using their device fingerprint, face unlock, or to use a hardware security key as an alternative to typing a password. Because it is directly phishing-resistant: the private key never leaves the user's device and is never sent anywhere and so there is no password that a fake login page can steal to gain access in the first place.

We did not make it the only way of logging in, and we didn't delete password login. We retained the Argon2id password hash function as the main mechanism, since creating a password hash function for every evaluator/tester and installing it on his/her hardware passkey would have complicated the demonstration. WebAuthn is not a password replacement system, it's an optional, more secure system that can operate in conjunction with passwords.

8. Before-and-After Security Testing

For every weakness, we tested the system before the control existed, and then we added the control, and then repeated the exact same test to confirm the behaviour actually changed. The table below summarises what we did for each one.

ID	Test Performed	Before (Unprotected)	After (Protected)
W1	In this we used one user's login token to request another user's balance.	Before the other user's private balance was returned with no error.	After the request is rejected with a 403 Forbidden response.
W2	In this we sent a transfer request with a fake "sender" field pointing to someone else's account.	Before the server debited the account named in the request body.	After the server ignores the body field and debits only the account that actually owns the login token.
W3	In this we attempted login with a wrong password, and separately checked what the error message revealed.	Before the error messages differed enough to reveal whether an email was registered; weak hashing would crack instantly if leaked.	After a single generic error is returned either way, stored hashes resist GPU cracking.
W4	In this we simulated a crash partway through a transfer.	Before the sender's balance was debited, but the receiver never received the funds.	After the whole operation rolls back automatically; no balance changes at all.
W5	In this we send the same transfer request twice in a row with the same idempotency key.	Before the two separate transfers were created, debiting the sender twice.	After the second request returns the original result; only one transfer is created.
W6	In this we attempted an unauthorized action and checked whether it was recorded anywhere.	Before nothing was written down; there was no way to see it had happened.	After a structured log entry is created with the timestamp, the action, and the outcome.
W7	In this we open the raw database file directly.	Before CNIC numbers were visible as plain, readable text.	After CNIC values appear as encrypted ciphertext; the UI shows only a masked version.
W8	In this we triggered a bulk export using a non-admin account, and separately exported a record containing a value starting with "=".	Before the full customer table could be downloaded in one request, and the "=" value would run as a formula when opened in a spreadsheet.	After non-admin accounts are blocked outright; admin exports are capped at 500 rows with personal fields masked and formula-triggering characters neutralized.
W9	In this we blocked an account from the admin view while that account was still logged in elsewhere.	Before, the blocked account could keep transferring money until its token expired naturally.	After the session is detected as blocked within about two seconds and then the user is logged out immediately.
W10	In this we sent a rapid burst of transfer requests, and separately tried a large transfer with only a normal login.	Before all requests succeeded with no slowdown or extra check, regardless of speed or amount.	After the requests beyond the rate limit are rejected, and also large transfers require the password to be re-entered.
W11	In this we tried to log in as a customer using only a captured or phished password.	Before a correct password alone was enough to get in, and there was no stronger option to choose.	Customers can register a passkey and also can sign in with biometrics or a hardware key; no password is typed or sent, so there is nothing to phish.

9. Security Design Table

This table follows the structure specified in the assignment brief, containing the asset, the weakness, its impact, the control we chose, where we implemented it, and the outcome after implementation.

ID	Asset	Weakness	Impact	Control	Where Implemented	Result
W1	Balance & history	No check that the caller owns the requested account (BOLA).	Unauthorized viewing of another customer's data.	Ownership guard comparing token ID to requested ID.	ownership.guard.ts	Before: data returned for any logged-in user. After: 403 for mismatched accounts.
W2	Wallet balance	Sender identity taken from client input.	Unauthorized debits from arbitrary accounts.	Sender always derived from the verified token.	wallet.service.ts	Before: body field controlled the debit. After: only the token holder's account is debited.
W3	Credentials	Weak/absent hashing and revealing login errors.	Credential leakage on breach; account enumeration.	Argon2id hashing + generic login errors.	auth.service.ts	Before: crackable hashes, revealing errors. After: strong hashing, uniform errors.
W4	Wallet balance	Debit and credit written as two separate operations.	Money lost or duplicated on a crash mid-transfer.	Single atomic database transaction.	wallet.service.ts	Before: partial writes possible. After: all-or-nothing transfer.
W5	Ledger	No protection against a repeated request.	Double-debiting from retried requests.	Idempotency key with cached response.	wallet.service.ts	Before: repeats created new transfers. After: repeats return the original result.
W6	Audit trail	Security events were not recorded.	No way to investigate incidents after the fact.	Structured audit logging (secrets excluded).	audit.service.ts	Before: silent failures and successes alike. After: every event is logged.
W7	Customer PII (CNIC)	Identity numbers stored as plain text.	Mass identity exposure if the database leaks.	AES-256-GCM encryption + on-screen masking.	crypto.util.ts	Before: CNIC visible in the raw database. After: stored as ciphertext, shown masked.
W8	Bulk export	No role check, volume cap, masking, or formula-injection protection on bulk export.	Mass data exfiltration in a single request (Santander pattern), plus possible CSV formula execution on open.	Admin-only, 500-row cap, masked output, CSV formula-injection sanitization.	admin export module	Before: unrestricted full-table export, unsanitized CSV cells. After: capped, masked, admin-only, formula-safe export.
W9	Account status	Blocking did not end an active session immediately.	A blocked account could keep transferring funds.	Status column + 2-second session polling.	AuthContext.tsx / schema.prisma	Before: active for up to 15 more minutes. After: logged out within ~2 seconds.
W10	Transfer API	No speed limit; large transfers needed only a normal login.	Rapid automated transfers; large unauthorized transfers.	Rate limiting + step-up password check above PKR 5,000.	wallet.controller.ts	Before: unlimited speed and no extra check. After: rate-limited, large

ID	Asset	Weakness	Impact	Control	Where Implemented	Result
						transfers need re-authentication.
W11	Login method	Passwords are the only way to prove identity.	Vulnerable to phishing and credential theft.	WebAuthn / FIDO2 passkey login, alongside password login.	auth module (WebAuthn)	Before: password-only. After: optional passwordless, phishing-resistant login available.

10. Individual Student Contributions

The individual contributions table is written below.

Student	Role	Main Contributions
Syed Mohid Ul Hassan (261-7709)	Lead Architecture, Frontend, Authentication & Data Protection	Designed the web dashboard using React 18 and API using NestJS. Implements secure login (W3), anti-enumeration (W1), ownership protection (W1), secure sender handling (W2), AES-256-GCM encryption (W7), database status kill-switch (W9), and 2-second session monitoring (W9), and FIDO2 WebAuthn passkeys (W11).
Muhammad Akash Afzal (261-7807)	Ledger, Audit, Exports & Risk Controls	Designed the Prisma database and atomic transactions (W4), the idempotency key storage (W5), Pino audit logging (W6), Security Testing Panel (W7, W8), customer statement and Santander bulk CSV export security (W10), and the rate limiting and step-up authentication for transactions over PKR 5,000.

11. Limitations and Possible Future Improvements

We did our best to be honest with ourselves as to what parts of this project were potentially beyond the scope of two people and which parts would need more polishing to be truly production worthy. Below are not items that we've missed by chance, but items that we have intentionally excluded.

- Encryption key storage: we use an environment variable for this key, which we are changing to AES-256-GCM. This should reside in a proper key-management service or hardware security module in a real implementation, and with the ability to rotate keys.
- Database choice: SQLite is chosen as it does not require any separate server and is easy to demonstrate. If this was a real money production system, it would require a database like PostgreSQL, or a database system with similar capabilities for concurrent access at scale.
- Log monitoring: our audit logs are organized and can be fed into a real monitoring platform (e.g. a SIEM) that will automatically correlate events and alert us, but we're not doing that now - someone will have to read our logs.
- Multi-factor authentication for all logins: WebAuthn is offered as an alternative stronger login method, but it was not mandatory to re-enter any authentication factor on every login, because this requires infrastructure (such as an SMS provider) which we did not have the time to implement for this assignment.
- Infrastructure-level protections: such as network segmentation, web application firewall and automated vulnerability scanning - was not part of the scope as this is a student deployment in a single machine environment and not a cloud production environment.
- Approval workflow for bulk exports: our export control limits volume and only allows us to run the bulk export as an admin; a production system would probably require a second person to sign off on the export before it could run at all.

Appendix A: Secure-System-Design Principles Mapping

The assignment brief asks us to relate our design back to the principles covered in class, where they genuinely apply. Rather than forcing every principle onto the system, the list below only includes the ones we can point to a real, specific piece of our implementation for.

Principle	Where It Shows Up In Our System
Confidentiality	CNIC data is encrypted using AES-256-GCM when stored. On screen, CNIC is shown in masked form only (W7). Transaction logs do not store passwords or other sensitive login information.
Integrity	The transfers are performed within a single atomic database transaction (W4), and the idempotency keys prevent being able to double-correct a balance for the same transfer twice (W5).
Availability	The Endpoint for transfer is rate-limited (W10), meaning that if many automated requests are made simultaneously, then no other requests are able to be made.
Authentication	The hashing of passwords is done using Argon2id (W3). WebAuthn passkeys provide an alternative authentication mechanism which is not vulnerable to phishing (W11).
Authorization	The ownership guard (W1) ensures that the user is able to read only his own balance and history, the admin role guard (W8) prevents ordinary users from bulk export.
Least Privilege	A normal customer account will not be able to access the audit log stream or bulk export feature, and an admin account will not be able to trigger a customer's transfer.
Secure Defaults	In the event of an unsuccessful login, the generic message is always returned, irrespective of the reason (W3); new accounts are not given what is essentially an 'active' status, but are given the 'active' status instead.
Separation of Responsibilities	Actions (such as blocking accounts, running exports) are not executed in the same part of the application as the transfer flow, so they don't impact each other.
Defense in Depth	Before moving money a request has to pass a few independent checks: token validity, rate limit, step-up check (for large amounts), ownership check, account-status check, and finally the atomic database write.
Reduction of Unnecessary Exposure	By limiting the amount of sensitive information displayed at once, CNIC masking, the 500-row export cap and omitting secrets from log output all reduce overall exposure of sensitive information.
Logging and Accountability	Each attempt to log in is logged as a structured log (W6) as is every transfer that is attempted, whether it is successful or not, and for history's sake, there is a record to return to if something needs investigation.
Resilience	A crash that occurs mid transfer does not leave money in an inconsistent state (W4), and a blocked account is logged out in approximately 2 seconds versus sitting in its token's active state (W9).

Some of the ideas mentioned in class, such as full operational resilience (two sets of infrastructure) or strict separation of duties (several levels of approval), apply partially here, because it is a student project and therefore not a production system. We have been clear about that in Section 11, and not boasting about something that we don't cover.

Appendix B: How to Run the Application Locally

Below are the instructions to run this application locally

Prerequisites

- Node.js v18 or later (check with: `node -v`)
- npm v9 or later (check with: `npm -v`)

Step 1 - Start the Backend API (NestJS + SQLite)

Open a terminal window in the project root directory and run:

- `cd backend`
- `npm install`
- `npx prisma generate`
- `npx prisma db push`
- `npx ts-node prisma/seed.ts` (creates the default admin account)
- `npm run dev`

The backend API will then be running at `http://localhost:3000`.

Step 2 - Start the Frontend Dashboard (React + Vite)

Open a second terminal window in the project root directory and run:

- `cd frontend`
- `npm install`
- `npm run dev`

The web dashboard will then be running at `http://localhost:5173`.

Step 3 - Open Prisma Studio (Optional Database Viewer)

Open a third terminal window inside the backend directory (`cd backend`) and run:

- `npx prisma studio`

Prisma Studio opens at `http://localhost:5555` and lets you inspect the raw SQLite tables directly.

Github: <https://github.com/MohidShah/FintechWallet>